

Phoenix: Verifiable Post-Quantum Voting with Malicious Ballot-Box Privacy

Navid Abapour and Constantin Cătălin Drăgan

Surrey Centre for Cyber Security, University of Surrey, UK
{n.abapour, c.dragan}@surrey.ac.uk

Abstract. E-voting systems must balance the voters’ need for privacy with the election’s goal of fairness and transparency, especially in a challenging and growing context like the post-quantum settings. While there are numerous protocols and systems with good voting guarantees under classical security settings (i.e., not post-quantum), there is a lack of substantial results for post-quantum schemes. All post-quantum e-voting protocols do not achieve satisfactory privacy, mostly due to informal or weak guarantees, while some offer a form of verifiability. Moreover, most schemes – even those specifically designed for online voting- do not offer the possibility of revoting. In this work, we propose **Phoenix**, the first post-quantum voting system that is both verifiable and strongly private (i.e., private even when the adversary controls the ballot box). As we design our scheme based on Fully Homomorphic Signatures, we further address a gap in the design of this cryptographic primitive by proposing an efficient and context-hiding instantiation; previous constructions are either efficient or context-hiding (due to the heavy use of trapdoors). We enhance our contribution by providing an implementation of **Phoenix** via standard BFV encryption in Microsoft SEAL library and IBM Research’s LaZer library for proof of knowledge and benchmarking **Phoenix** on different CPUs.

1 Introduction

The idea of electronic voting is to modernize democratic processes through more accessibility and efficiency [1]; however, apart from the fact that people trust more in simple election systems with easily understandable procedures than complicated ones [2], designing voting schemes that can meet a wide range of security and privacy requirements remains one of the challenging problems in cryptography [3]. The paradox lies between transparency toward public verifiability and a high level of privacy for individual votes [2], as the level of privacy decreases by increasing verifiability and vice versa. This challenge gets more complicated by the advent of quantum computers, which will be in a position to retroactively break today’s classically encrypted ballots [4].

Transition to post-quantum (PQ) e-voting needs to happen as soon as possible, because of the Store Now, Decrypt Later attack [5], but migration from classical cryptosystems to post-quantum ones is not trivial and all applications face many issues for this transition [6], from the substantial increase in key and ciphertext sizes imposed by post-quantum schemes [7] to interoperability and backward compatibility challenges arising during the coexistence of classical and quantum-resistant algorithms [8]. However, in voting systems, apart from these issues, this migration has even extra challenges, as mentioned earlier on balancing privacy and verifiability.

It is expected a voting system to have privacy [9] (no entity can learn the content of ballots) and verifiability [10–13] (voters can verify their ballots were recorded and anyone

can verify correct tallying), so the current post-quantum electronic voting schemes need to go beyond innovations in building block and consider bigger picture by taking account formal cryptographic privacy and verifiability properties in voting, because elections are more than just counting ballots [1]. No matter how much novel and efficient a new building block for voting scheme is, it will not be securely-constructive if the resulting voting system can not satisfy cryptographic privacy definitions (e.g., ballot privacy [9]). While studying post-quantum voting schemes [4, 14–23], we observed that if a traditional voting system is ballot-private, it is not straightforward to instantiate a post-quantum variant of it [4], and just replacing its classical building blocks with post-quantum ones does not necessarily make the resulting voting system ballot-private. This is because, even when retaining the same architecture, migrating to post-quantum primitives (I) alters the underlying computational assumptions, thereby affecting information flow, privacy guarantees, and trust models [14]; and (II) introduces building blocks that, despite preserving classical properties (e.g., homomorphism), may possess additional attributes [4] capable of leaking information, e.g., the design ideas of discrete log-based voting protocols do not carry over easily to the lattice setting, due to specific problems [24] such as noise growth.

As summarized in Table 1, our study reveals a consistent pattern: no existing scheme simultaneously achieves strong ballot privacy, formal verifiability, and practical efficiency under post-quantum assumptions. Revoting support [25], important for usability and coercion mitigation, is similarly absent from all but one [19] scheme.

As part of our endeavor to fill aforementioned gaps by designing a new voting system, while we were using Fully Homomorphic Signature (FHS) in it, we noticed that the existing FHS miss a balance of efficiency and context-hiding property (revealing no info about signed data while remaining verifiable); Gorbunov et al.’s context-hiding FHS [26] requires $O(N)$ trapdoor samples per N -element dataset, whereas Luo et al.’s efficient construction [27] eliminates this cost but lacks context-hiding. No existing FHS provides both, yet without context-hiding a derived tally signature may leak individual ballot information.

1.1 Contributions

To address the gaps identified above, in this work we build **Phoenix**, the first PQ verifiable voting scheme that achieves delay-use malicious-ballot-box privacy [28], together with strong correctness¹, strong consistency, ideal functionality [9], and strong verifiability (individual, universal, and eligibility) [12], all under standard lattice assumptions, while also supporting revoting. As part of the design of **Phoenix**, we further construct the first efficient leveled fully homomorphic signature scheme with context-hiding, by building on the efficient evaluation of Luo et al. [27] and incorporating a context-hiding mechanism via the **SampleLeft–SampleRight** paradigm of Gorbunov et al. [26]. Eventually, we provide an implementation of **Phoenix** [29] and benchmark ballot creation across four platforms to demonstrate that end-to-end ballot creation completes in under 70 ms even for elections with up to 2^{25} candidates.

1.2 Related Works

The voting systems that are resistant to quantum attacks have diverse structures with minimal privacy and/or verifiability guarantees, where Table 1 summarizes our study.

¹ Individual verifiability: Voter confirms ballot was recorded. Universal verifiability: Anyone can verify correct tallying [10–13]. Strong correctness: Honest ballots are never rejected [9]. Strong consistency: Valid ballots tally correctly.

As a note, there are two main methods of election tally [30,31]: *Homomorphic* aggregates encrypted votes directly into an encrypted sum that is decrypted once, while *mix-net* anonymizes individual ciphertexts through a shuffle before decrypting each ballot.

Starting with Epoque [14], it splits each vote into secret shares, commits to each share, encrypts the shares under an IBE scheme (binding them to the voter’s identity) and sends them to multiple trustees, who tally via homomorphic addition of commitments verified by a ZKP of well-formedness; all trustees must be online and actively participate in both the ballot weeding and tallying phases. Epoque’s privacy proof uses a privacy definition called *Vote Privacy* [32] that Bernhard et al. [9] shows this is a weak definition of ballot privacy since it restricts the class of protocols, may yield inconsistent results under natural result functions (e.g., majority [33]), fails to apply to protocols outputting auxiliary data (e.g., tally proofs in ElectionGuard [34]), and misses privacy violations beyond same-vote detection [9]. Moreover, their Lemma 5 [31] omits the full hybrid argument for secret-share uniformity under commitment hiding. Also, Cortier et al. in [12] shows why the Epoque’s verifiability definition [13] is weak, since it does not distinguish ballot deletion from ballot modification [12]. Moreover, concrete post-quantum parameters for all building blocks are also absent in Epoque.

Hough et al. proposed NTRU Mix-Net [15], in which votes are NTRU-encrypted, anonymized through a verifiable mix-net, and recovered via threshold distributed decryption, with ZKPs ensuring honest execution at each stage. In this scheme, NTRU parameters must be carefully chosen to avoid the overstretched regime where $q > 0.0058 \cdot \sigma^2 \cdot d^{2.484}$ (with d the polynomial degree, q the modulus, and σ the noise distribution width) enables subfield lattice attacks, and the scheme’s privacy fundamentally relies on the assumption that at least one mix server remains honest without providing detection mechanisms for malicious mixing beyond proof verification. It is not straightforward how the privacy claims in [15] are achievable; although Hough et al. [15] does not propose a complete voting scheme, upon using their NTRU Mix-Net, they refer the privacy of resulting system to another mix-net by Aranha et al. [24], but even their theoretical system (full version’s Appendix H [30]) does not provide any proofs or formal guarantees.

In FOO-based Voting [4], a voter encrypts their vote, gets a threshold blind signature on it from multiple authorities, and anonymously posts it to a bulletin board, where authorities later reveal decryption shares so anyone can tally. It combines threshold lattice-based blind signatures [35–37] with Dual-Regev encryption [38,39], but one-more unforgeability of the underlying blind signature remains conjectural [40], and privacy breaks if the required perfectly anonymous submission channel is compromised.

Code-based Voting [16], the first code-based verifiable voting system, relies on the less-studied dual-LPN assumption whose cryptanalytic background is limited, concrete 128-bit parameters against ISD attacks remain uncertain [41], and ballot privacy claims lack formal proofs.

In EVOLVE [17], each vote is split into additive shares, committed to multiple authorities using a lattice-based homomorphic commitment, with an OR-proof ensuring vote validity and amortized ZKPs by authorities enabling homomorphic tallying; it is post-quantum secure but lacks individual verifiability, requires a large voter threshold for efficiency, and supports no revoting. EVOLVE modifies the BPRIV definition [9] but the adaptation sidesteps the simulator requirement by making the tally consistent with the board in both worlds; in standard BPRIV, $\beta=1$ requires a simulator Sim to produce a fake proof π' consistent with BB_1 , which EVOLVE never constructs, and whether their notion implies standard BPRIV is not formally analyzed. EVOLVE also proves only a

Scheme	Assumptions	Building Blocks	Privacy	Verifiability	Revoting ²
Epoque [14]	RLWE, RSIS	IBE [43], Com [44], NIZK [45]	Weak BPRIV [32]	indiv, univ	×
NTRU-Mix [15]	RLWE, NTRU	PKE [46], Com [47], ZKP [48–50]	×	univ	×
FOO-based [4]	RLWE, RSIS	TPKE [38, 39], BSign [35–37]	×	partial univ	×
Code-based [16]	Dual-LPN	PKE [51, 52], Com [16], NIZK [16, 53]	×	indiv, univ	×
EVOLVE [17]	MLWE, MSIS	PKE [17], ZKP [54], Sig [55], Com [56]	BPRIV [9]	partial univ	×
EVOLVED [18]	MLWE, MSIS	PKE [18], Com [17], HW-Proof [57]	×	partial univ	×
Homo-LWE [19]	LWE	FHE [58], Sign [59], NIZK [60]	BPRIV [9]	indiv, univ	✓
BPRIV-BC [20]	MLWE, MSIS	Com [61], PostSec [62], NIBSign [63]	BPRIV [9]	indiv, univ	×
Kyber-BC [21]	MLWE	KEM [64], Block [65, 66], Hash [67]	×	partial indiv	×
Hybrid-BC [22]	ECDH	ABE [22], ECDH [68], Block [69, 70]	×	partial indiv	×
Steg-MQPC [23]	MQ	MQPC [71, 72], LSB-Steg [23]	×	×	×
Phoenix [Ours]	RLWE, SIS	FHE [73], ZKP [74], FHS Def. 12	du-mb-BPRIV [28]	indiv, univ	✓

Table 1: Comparison of post-quantum electronic voting schemes based on cryptographic privacy properties with formal proofs providing and features they guarantee. du-mb-BPRIV: the ballot box cannot learn individual votes even when it is malicious and verification happens after tallying. Here, consider zero-knowledge proof ZKP and non-interactive one as NIZK, hamming weight proof HW-Proof, multivariate quadratic polynomial cryptography MQPC, identity-based encryption IBE, commitment Com, public-key encryption PKE, blind signature BSign, threshold public-key encryption TPKE, digital signature Sig, fully homomorphic encryption FHE, polynomial commitment PCom, posterior security PostSec, non-interactive blind signature NIBSign, key encapsulation mechanism KEM, blockchain Block, hash function Hash, elliptic-curve Diffie–Hellman ECDH, attribute-based encryption ABE, least-significant-bit steganography LSB-Steg, and fully homomorphic signature FHS.

weaker “realistic result” consistency rather than strong consistency [9]. EVOLVED [18] extends EVOLVE with Hamming Weight Proofs but inherits the same definitional gaps.

Chillotti et al. [19] present a homomorphic LWE-based voting scheme but provide only a proof sketch for ballot privacy. Abapour et al. [20] give a complete ballot privacy proof, but the scheme is exclusively for economic structures and, due to its blockchain dependency, lacks the flexibility for general use [42]; the same limitation applies to the blockchain-based schemes of Sekar et al. [21] and Mahanayak et al. [22] as well.

2 Preliminaries

Throughout this work, we use standard notation as λ denotes the security parameter; $[n]$ denotes the set $\{1, \dots, n\}$; $\|\cdot\|$ and $\|\cdot\|_\infty$ denote the Euclidean and infinity norms of a vector, respectively; $\tilde{\mathbf{T}}$ denotes the Gram–Schmidt orthogonalization of a matrix $\mathbf{T}$.

2.1 Computational Assumptions

Let n, m, q be positive integers with q prime, $R_q = \mathbb{Z}_q[X]/(f(X))$ for an irreducible f of degree n , and χ a distribution over $\mathbb{Z}_q$.

SIS and MSIS [75, 76]. Given $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$, $\text{SIS}_{n,m,q,\beta}$ asks to find a non-zero $\mathbf{z} \in \mathbb{Z}^m$ with $\mathbf{A}\mathbf{z} = \mathbf{0} \pmod{q}$ and $\|\mathbf{z}\| \leq \beta$. Its module variant $\text{MSIS}_{n,k,m,q,\beta}$ asks for a non-zero $\mathbf{z} \in R_q^m$ with $\mathbf{A}\mathbf{z} = \mathbf{0}$ in R_q^k and $\|\mathbf{z}\| \leq \beta$, given $\mathbf{A} \leftarrow \mathbb{Z}_q^{k \times m}$.

RLWE and MLWE [77, 78]. $\text{RLWE}_{n,q,\chi}$ asks to distinguish $\{(a_i, a_i s + e_i)\}$ from $\{(a_i, u_i)\}$ for $a_i, u_i \leftarrow \mathbb{Z}_q$, $s \leftarrow \mathbb{Z}_q$, $e_i \leftarrow \chi$. Its module variant $\text{MLWE}_{n,k,\ell,q,\chi}$ asks to distinguish $(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e})$ from $(\mathbf{A}, \mathbf{u})$ for $\mathbf{A} \leftarrow \mathbb{Z}_q^{k \times \ell}$, $\mathbf{s} \leftarrow \chi^\ell$, $\mathbf{e} \leftarrow \chi^k$, $\mathbf{u} \leftarrow \mathbb{Z}_q^k$.

² A *revote policy* Policy is a function that, given a list of ballots, selects which ballot to count per voter. Two common instantiations are: (I) only the first valid ballot cast by each voter is retained, (II) only the last valid ballot cast by each voter is retained (which Phoenix focuses on it).

2.2 Building Blocks

We now state the cryptographic primitives that our Phoenix builds upon.

Definition 1 (Hash Function [79]). A cryptographic hash function $H: \{0,1\}^* \rightarrow \{0,1\}^\lambda$ is a deterministic function that maps arbitrary-length inputs to fixed-length outputs of λ bits, satisfying:

- For any PPT adversary $\mathcal{A}$, $\Pr[(x, x') \leftarrow \mathcal{A}(1^\lambda) : x \neq x' \wedge H(x) = H(x')] \leq \text{negl}(\lambda)$ (Collision Resistance).
- For random $y \in \{0,1\}^\lambda$ and PPT adversary $\mathcal{A}$, $\Pr[x \leftarrow \mathcal{A}(y) : H(x) = y] \leq \text{negl}(\lambda)$ (Preimage Resistance).

Definition 2 (BFV Encryption [73]). The BFV scheme is a leveled homomorphic encryption scheme whose security relies on the hardness of the RLWE problem. $\text{BFV} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Add}, \text{Rerand})$ consists of:

- $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$: Outputs public encryption key pk and secret decryption sk .
- $c \leftarrow \text{Enc}(pk, \mu)$: Encrypts plaintext μ under pk and outputs ciphertext c .
- $\mu \leftarrow \text{Dec}(sk, c)$: Decrypts ciphertext c using sk and recovers plaintext μ .
- $c \leftarrow \text{Add}(c_1, c_2)$: Homomorphically adds two ciphertexts, which satisfies the relation of $\text{Dec}(sk, \text{Add}(c_1, c_2)) = \text{Dec}(sk, c_1) + \text{Dec}(sk, c_2)$.
- $c' \leftarrow \text{Rerand}(pk, c)$: Re-randomises c to produce a fresh ciphertext c' satisfying $\text{Dec}(sk, c') = \text{Dec}(sk, c)$, indistinguishable from a fresh encryption of same plaintext.

Based on [80, 81], BFV satisfies: *correctness* ($\text{Dec}(sk, \text{Enc}(pk, \mu)) = \mu$ with overwhelming probability for all $\mu \in R_t$, provided noise stays below the decryption threshold); *IND-CPA security* under the $\text{RLWE}_{n,q,\chi_{\text{err}}}$ ($|\Pr[\mathcal{A}(pk, \text{Enc}(pk, \mu_0))=1] - \Pr[\mathcal{A}(pk, \text{Enc}(pk, \mu_1))=1]| \leq \text{negl}(\lambda)$ for all PPT $\mathcal{A}$, $\mu_0, \mu_1 \in R_t$); and BFV satisfies the *additive homomorphism* ($\text{Dec}(sk, \text{Add}(\text{Enc}(pk, \mu_1), \text{Enc}(pk, \mu_2))) = \mu_1 + \mu_2 \pmod{t}$).

Definition 3 (Lattice-Based Zero-Knowledge Proof System [74]). The proof system ZKP, based on the hardness of MSIS and MLWE, proves knowledge of a witness satisfying a specified relation $\mathcal{R}$. $\text{ZKP} = (\text{Setup}, \text{Prove}, \text{Verify})$ consists of:

- $pp \leftarrow \text{Setup}(1^\lambda)$: Outputs public parameters pp defining the relation $\mathcal{R}$.
- $\pi \leftarrow \text{Prove}(pp, x, w)$: Given a statement x and witness w such that $(x, w) \in \mathcal{R}$, outputs a proof π via the Fiat–Shamir transform.
- $\{0,1\} \leftarrow \text{Verify}(pp, x, \pi)$: Accepts if π is a valid proof that the prover knows a witness w such that $(x, w) \in \mathcal{R}$, and rejects otherwise.

Here, proved relation is $\mathcal{R}_{\text{vote}} = \{((c, pk, upk), (v, s_1, e, s_2)) \mid c = \text{BFV.Enc}(pk, v; s_1, e, s_2) \wedge v \in \{0,1\} \wedge \|s_1\|, \|e\|, \|s_2\| \leq \beta_{\text{enc}}\}$, where the witness is the secret bit v and the randomness-error triple (s_1, e, s_2) bounded by the BFV noise parameter β_{enc} .

Via the Fiat–Shamir transform, all ZKPs are non-interactive (secure in ROM). Considering [74, 82–84], proof system satisfies: *completeness* (Verify accepts an honest proof for any valid witness (s_1, m, s_2) w.o.p.); *knowledge soundness* under MSIS (any PPT adversary producing an accepting proof for a false statement yields, via the ABDLOP extractor [74], a short vector breaking MSIS with non-negligible probability); and *computational zero-knowledge* under MLWE (a PPT simulator produces transcripts computationally indistinguishable from real proofs, so π leaks nothing beyond the truth of the asserted relations).

Definition 4 (Fully Homomorphic Signature). Let $\ell, d_{\max} \in \mathbb{N}$ denote the maximum dataset size, and maximum supported circuit depth, respectively, and let $\mathcal{C}$ be a class of boolean circuits of depth at most $d_{\max}$ over NAND (universal boolean gates). The leveled fully homomorphic signature FHS consists of PPT algorithms as:

- $(pk, sk) \leftarrow \text{Setup}(1^\lambda, 1^{|\tau|}, 1^\ell, 1^{d_{\max}})$: Given security parameter and bounds on tag length, dataset size, circuit depth, outputs a public key pk and a signing trapdoor sk .
- $vk \leftarrow \text{KeyGen}(pk, \tau)$: Given pk and a dataset tag $\tau \in \{0,1\}^{|\tau|}$, derives a per-dataset verification key vk .
- $\sigma_i \leftarrow \text{Sign}(sk, vk, \tau, \mu_i, i)$: Given trapdoor sk , verification key vk , tag τ , message bit $\mu_i \in \{0,1\}$, index $i \in [\ell]$, outputs signature σ_i authenticating dataset's i -th entry.
- $\sigma_f \leftarrow \text{Eval}(vk, \tau, \mu, \sigma, f)$: Given vk , tag τ , message vector $\mu \in \{0,1\}^\ell$, base signatures σ , and a circuit $f \in \mathcal{C}$, outputs an evaluated signature σ_f authenticating $f(\mu)$.
- $\{0,1\} \leftarrow \text{Verify}(vk, \tau, \sigma_f, f)$: Accepts if σ_f is a valid signature on $f(\mu)$ under tag τ , and rejects otherwise.
- $\tilde{\sigma}_f \leftarrow \text{Hide}(vk, y, \sigma_f)$: Given vk , output value $y = f(\mu)$, evaluated signature σ_f , re-randomizes σ_f into context-hiding signature $\tilde{\sigma}_f$ that reveals nothing beyond y .
- $\{0,1\} \leftarrow \text{HVerify}(vk, \tau, \tilde{\sigma}_f, y, f)$: Accepts if $\tilde{\sigma}_f$ is a valid context-hiding signature certifying that f evaluated to y under tag τ , and rejects otherwise.

Based on [26, 27, 85], we consider the following properties for FHS with a focus on unforgeability and context-hiding.

Signing Correctness. For all λ , $(pk, sk) \leftarrow \text{Setup}(1^\lambda)$, $vk \leftarrow \text{KeyGen}(pk, \tau)$, $\mu_i \in \{0,1\}$, $i \in [\ell]$: $\Pr[\text{Verify}(vk, \tau, \text{Sign}(sk, vk, \tau, \mu_i, i), \text{id}_i) = 1] \geq 1 - \text{negl}(\lambda)$.

Evaluation Correctness. For all λ , $(pk, sk) \leftarrow \text{Setup}(1^\lambda)$, $vk \leftarrow \text{KeyGen}(pk, \tau)$, $\mu \in \{0,1\}^\ell$, $f \in \mathcal{C}$, and base signatures σ produced by Sign: $\Pr[\text{Verify}(vk, \tau, \text{Eval}(vk, \tau, \mu, \sigma, f), f) = 1] \geq 1 - \text{negl}(\lambda)$.

Succinctness. For all λ , $(pk, sk) \leftarrow \text{Setup}$, $vk \leftarrow \text{KeyGen}(pk, \tau)$, $\mu \in \{0,1\}^\ell$, $f \in \mathcal{C}$, and $\sigma_f \leftarrow \text{Eval}(vk, \tau, \mu, \sigma, f)$: $\|\sigma_f\| \leq p(\lambda) \cdot \log \ell$ for a fixed polynomial p , independent of $|f|$.

Definition 5 (Existential Unforgeability under Fully Adaptive Chosen-Message Attacks). We define the security of the FHS scheme via the following experiment $\text{Expt}_A^{\text{EU-F-ACMA}}(1^\lambda)$ between a challenger and a PPT adversary $\mathcal{A}$:

1. **Setup.** The challenger runs $(pk, sk) \leftarrow \text{Setup}(1^\lambda, 1^{|\tau|}, 1^\ell, 1^{d_{\max}})$ and sends pk to $\mathcal{A}$.
2. **Signing Queries.** The adversary $\mathcal{A}$ adaptively submits queries of the form (τ, μ_i, i) , where the tag $\tau \in \{0,1\}^{|\tau| \geq \lambda}$ is a dataset identifier, $\mu_i \in \{0,1\}$, and $i \in [\ell]$. The challenger maintains a list T_τ for each queried tag τ and responds with $\sigma_i \leftarrow \text{Sign}(sk, vk, \tau, \mu_i, i)$, adding (μ_i, i) to T_τ (aborting if $|T_\tau| > \ell$ or if a different message was previously signed at the same index).
3. **Forgery.** The adversary $\mathcal{A}$ outputs a tuple $(\tau^*, \sigma_{f^*}^*, \mu^*, f^*)$. The experiment outputs 1 if and only if $\text{Verify}(vk^*, \tau^*, \sigma_{f^*}^*, f^*) = 1$ (or $\text{HVerify}(vk^*, \tau^*, \tilde{\sigma}_{f^*}^*, \mu^*, f^*) = 1$ when context hiding is applied) and one of the following holds:
 - Type I forgery: $\tau^* \neq \tau_j$ for any previously queried tag τ_j .
 - Type II forgery: $\tau^* = \tau_j$ for some queried tag τ_j , and $\mu^* \neq f^*(\mu_j)$, where μ_j is the message vector signed under τ_j .

We say that the FHS scheme is existentially unforgeable against fully adaptive chosen-message attacks if for all PPT adversaries $\mathcal{A}$, $\Pr[\text{Expt}_{\mathcal{A}}^{\text{EU-F-ACMA}}(1^\lambda) = 1] \leq \text{negl}(\lambda)$.

Definition 6 (Context-Hiding Security). We say that the FHS scheme supports statistical context hiding if there exist additional PPT procedures $\tilde{\sigma}_f \leftarrow \text{Hide}(vk, y, \sigma_f)$ and $\text{HVerify}(vk, \tau, \tilde{\sigma}_f, y, f)$ such that the following three properties hold:

1. **Correctness of Hiding.** For any $(pk, sk) \in \text{Setup}(1^\lambda, 1^{|\tau|}, 1^\ell, 1^{d_{\max}})$, $vk \in \text{KeyGen}(pk, \tau)$, and any (σ_f, y, f) such that $\text{Verify}(vk, \tau, \sigma_f, f) = 1$, for any $\tilde{\sigma}_f \in \text{Hide}(vk, y, \sigma_f)$ we have $\text{HVerify}(vk, \tau, \tilde{\sigma}_f, y, f) = 1$.
2. **Unforgeability.** The unforgeability guarantee of Definition 5 continues to hold when the Verify procedure in the security experiment is replaced by HVerify.
3. **Statistical Indistinguishability.** There exists a PPT simulator Sim such that for any fixed (worst-case) choice of $(pk, sk) \in \text{Setup}(1^\lambda, 1^{|\tau|}, 1^\ell, 1^{d_{\max}})$, $vk \in \text{KeyGen}(pk, \tau)$, and any (σ_f, y, f) satisfying $\text{Verify}(vk, \tau, \sigma_f, f) = 1$, we have $\text{Hide}(vk, y, \sigma_f) \stackrel{\text{stat}}{\approx} \text{Sim}(sk, vk, \tau, y, f)$, where the randomness is only over the coins of Hide and Sim. In particular, the dataset tag component τ in the verification key vk depends only on (sk, τ) and not on the data $\{\mu_i\}$.

3 Post-Quantum Verifiable du-mb-BPRIV Voting System

Entities. Our voting system, Phoenix, involves four distinct roles. A *Registrar* generates and privately distributes signing keys to voters, sending the corresponding verification keys to the server. *Voters* encrypt and sign their ballots, submitting them through an authenticated channel, with only their latest ballot retained if they re-vote. The *Voting Server* maintains the *Public Bulletin Board* (PBB), validating and recording all accepted ballots. Finally, a group of decryption *trustees* collectively hold the election's private key under a threshold scheme, so that only a required minimum number of them need to participate to decrypt the final result.

Voting Protocol. In Phoenix, voters encrypt their vote under the election public key, prove its well-formedness via a ZKP, and sign the resulting ballot with their credential key, after which the server validates and records it on the BB for homomorphic tallying. The Phoenix consists of the following procedures.

- $\text{Setup}(1^\lambda)$: Runs $(pk, sk) \leftarrow \text{BFV.KeyGen}(1^\lambda)$ and $pp \leftarrow \text{ZKP.Setup}(1^\lambda)$. Returns the election key pair (pk, sk) and public parameters pp .
- $\text{Register}(id)$: Runs $(pk_{fhs}, sk_{fhs}) \leftarrow \text{FHS.Setup}(1^\lambda, 1^{|\tau|}, 1^\ell, 1^{d_{\max}})$, draws a fresh per-voter tag $\tau \leftarrow \{0, 1\}^{|\tau|}$ independently of id , and sets $upk \leftarrow (\text{FHS.KeyGen}(pk_{fhs}, \tau), \tau)$ and $usk \leftarrow sk_{fhs}$. The tag is bundled into upk so that it passes de-identification and remains available on pbb for FHS.Verify ³.

³ Here, τ is the FHS dataset tag of Definition 4, instantiated per-voter; bundling it into upk makes sure it remains available on pbb (where id has been removed) for subsequent FHS.Verify calls in *Valid* and *Tally*.

- $\text{Vote}(id, v, pk, upk, usk)$: The voter parses $upk = (vk, \tau)$, encrypts the vote as $c \leftarrow \text{BFV.Enc}(pk, v)$, produces a proof $\sigma \leftarrow \text{ZKP.Prove}(pp, (c, upk), (v, s_1, e, s_2))$ binding the statement to upk (preventing cross-voter replay), and signs the ciphertext–proof pair $s \leftarrow \text{FHS.Sign}(usk, vk, \tau, (c, \sigma), 1)$. The signature is over the ballot contents alone and is independent of id , so that ballots remain publicly verifiable on pbb after de-identification. Returns $b = (id, upk, c, \sigma, s)$; id appears only in the envelope, for the server’s access-control check in Valid , and is removed by remID before publication.
- $\text{Valid}(BB, b, pk)$: Parses $b = (id, upk, c, \sigma, s)$ and verifies three conditions: (1) no ballot in BB pairs id with a different credential, or uses upk for a different identity (i.e., each identity maps to exactly one credential, preventing double voting under different keys); (2) the zero-knowledge proof is valid: $\text{ZKP.Verify}(pp, (c, upk), \sigma) = 1$, ensuring the encrypted vote is well-formed; (3) signature is valid: $\text{FHS.Verify}(vk, \tau, s, (c, \sigma))$, confirming the ballot was created by the legitimate credential holder. Returns **true** iff all three conditions hold.
- $\text{Publish}(BB)$: Publishes each ballot together with its hash fingerprint: $pbb = \{(b, H(b)) \mid b \in C\}$. Voter identities are removed from published ballots to protect anonymity, where $\text{remID}(b)$ removes the voter identity id from ballot $b = (id, upk, c, \sigma, s)$, yielding $\tilde{b} = (upk, c, \sigma, s)$, so $pbb = \{(\tilde{b}, H(\tilde{b})) \mid \tilde{b} = \text{remID}(b), b \in C\}$.
- $\text{Tally}(BB, sk)$: The server first retains, for each credential upk , only its most recent entry in BB , yielding BB^* ; this realises re-voting and bounds each voter’s contribution to one ballot. It then verifies every $b \in BB^*$ with ZKP.Verify and FHS.Verify , collecting the ciphertexts of accepted ballots in vbb . Using the additive homomorphism of BFV, it computes $c^* \leftarrow \text{BFV.Add}(vbb)$ and decrypts $r \leftarrow \text{BFV.Dec}(sk, c^*)$. Finally, it derives the public view $pbb \leftarrow \text{Publish}(BB)$ and produces a zero-knowledge proof of correct tallying $\pi \leftarrow \text{ZKP.Prove}(pp, (pk, pbb, r), sk)$ certifying that r matches the published board. Returns (r, π) .
- $\text{Verify}((pk, pbb, r), \pi)$: Checks the tally proof by running $\text{ZKP.Verify}(pp, (pk, pbb, r), \pi)$ and returns its boolean output, allowing any third party to confirm that the published result r correctly reflects the contents of the public bulletin board pbb .
- $\text{VerifyVote}(b, pbb)$: Allows a voter to confirm their ballot was correctly recorded by checking whether $H(b) \in \{h \mid (b', h) \in pbb\}$. Using a hash fingerprint rather than the full ballot makes the check lightweight, while collision resistance of H ensures the check is sound.

Note that sk and usk are treated as single secrets above; in practice, both can be threshold-shared among multiple trustees and registrars respectively [86], without affecting the security proofs. Phoenix supports re-voting as if a voter submits a new ballot, only the most recent one is retained on the bulletin board.

Phoenix uses FHS on singleton datasets ($\ell = 1$): a ballot is signed, then later verified, with no intermediate evaluation. For a message $M \in \{0, 1\}^{\ell_M}$ we therefore write $s \leftarrow \text{FHS.Sign}(usk, vk, \tau, M, 1)$ as shorthand for bit-wise signing of M followed by FHS.Eval on the identity circuit f_M that outputs M ; and $\text{FHS.Verify}(vk, \tau, s, M)$ for the matching call $\text{FHS.Verify}(vk, \tau, s, f_M)$ of Definition 4; both calls unfold to the 5-input/4-input of Definition 4.

Setup(1^λ)	Vote(id, v, pk, upk, usk)	Tally(BB, sk)
1: $(pk, sk) \leftarrow \text{BFV.KeyGen}(1^\lambda)$ 2: $pp \leftarrow \text{ZKP.Setup}(1^\lambda)$ 3: return (pk, sk, pp)	1: $(vk, \tau) \leftarrow upk$ 2: $c \leftarrow \text{BFV.Enc}(pk, v)$ 3: $\sigma \leftarrow \text{ZKP.Prove}(pp, (c, upk), (v, s_1, e, s_2))$ 4: $s \leftarrow \text{FHS.Sign}(usk, vk, \tau, (c, \sigma), 1)$ 5: return $b \leftarrow (id, upk, c, \sigma, s)$	1: $vbb \leftarrow []$ 2: $BB^* \leftarrow \{b \in BB \mid b \text{ is the latest entry for its } upk\}$ 3: for $b \in BB^*$ do 4: $(id, upk, c, \sigma, s) \leftarrow b$ 5: $(vk, \tau) \leftarrow upk$ 6: $e_1 \leftarrow \text{ZKP.Verify}(pp, (c, upk), \sigma)$ 7: $e_2 \leftarrow \text{FHS.Verify}(vk, \tau, s, (c, \sigma))$ 8: if $(e_1 \wedge e_2)$ then $vbb \leftarrow vbb \cup \{c\}$ 9: $c^* \leftarrow \text{BFV.Add}(vbb)$ 10: $r \leftarrow \text{BFV.Dec}(sk, c^*)$ 11: $pbb \leftarrow \text{Publish}(BB)$ 12: $\pi \leftarrow \text{ZKP.Prove}(pp, (pk, pbb, r), sk)$ 13: return (r, π)
Register(id)	Valid(BB, b, pk)	
1: $\text{tmp} \leftarrow \text{FHS.Setup}(1^\lambda, 1^{ \tau }, 1^\ell, 1^{d_{\max}})$ 2: $(pk_{fhs}, sk_{fhs}) \leftarrow \text{tmp}$ 3: $\tau \leftarrow \{0, 1\}^{ \tau }$ 4: $upk \leftarrow (\text{FHS.KeyGen}(pk_{fhs}, \tau), \tau)$ 5: $usk \leftarrow sk_{fhs}$ 6: return (upk, usk)	1: $(id, upk, c, \sigma, s) \leftarrow b$ 2: $e_1 \leftarrow \forall x, y. (x, y, \cdot, \cdot) \in BB \Rightarrow (x, y) = (id, upk) \vee (x \neq id \wedge y \neq upk)$ 3: $e_2 \leftarrow \text{ZKP.Verify}(pp, (c, upk), \sigma)$ 4: $e_3 \leftarrow \text{FHS.Verify}(vk, \tau, s, (c, \sigma))$ 5: return $(e_1 \wedge e_2 \wedge e_3)$	
Verify($(pk, pbb, r), \pi$)	Publish(BB)	
1: return $\text{ZKP.Verify}(pp, (pk, pbb, r), \pi)$	1: $pbb \leftarrow \{(\tilde{b}, H(\tilde{b})) \mid \tilde{b} \leftarrow \text{remID}(b)\}$ 2: return pbb	
VerifyVote(b, pbb)		
1: $\tilde{b} \leftarrow \text{remID}(b)$ 2: return $H(\tilde{b}) \in \{h \mid (\cdot, h) \in pbb\}$		

Fig. 1: The Phoenix voting system built upon $\text{BFV} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Add})$, $\text{ZKP} = (\text{Setup}, \text{Prove}, \text{Verify})$, $\text{FHS} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$, and hash function H . The function $\text{remID}(b)$ removes the voter identity from ballot b .

3.1 Concrete Parameters

Phoenix targets $\lambda = 128$ bits of post-quantum security throughout. H is instantiated with SHA-256 (256-bit output, ≥ 128 -bit collision resistance, unaffected by Grover’s algorithm beyond a factor-two preimage reduction [87]). For BFV, we adopt the Homomorphic Encryption Standard [88] parameter set $(n = 4096, \log q = 109, t = 2)$, achieving ≈ 128 -bit security under $\text{RLWE}_{n,q,\chi}$ per the lattice estimator [89]. For ZKP, we follow the ABDLOP instantiation of [74] with $d = 4, n = 64, \log q \approx 32$, challenge space 2^{256} , and $\beta = 2^{14}$, achieving 128-bit soundness under $\text{MSIS}_{d,m,\beta'}$ and $\text{MLWE}_{d,d,\chi}$. For FHS, we set $n = 512, m = 2n \lceil \log q \rceil \approx 22,528, \log q = 22$, with verification bound B derived from $s \geq \|\mathbf{T}_{\mathbf{A}^*}\| \cdot \omega(\sqrt{\log m})$ [90, 91], context-hiding bound $\tilde{\beta}_{\max} = \beta_{\max} \cdot p_{\text{extra}}(n, m, \log q)$ (Definition 12), and $d_{\max} = 30$ sufficient for Phoenix’s tallying circuits.

3.2 Performance and Efficiency Analysis

None of the existing PQ voting schemes [4, 14, 16–23] provided any implementation, except [15]. The core implementation of Phoenix is available in [29]; we benchmarked the Phoenix voting client, which encompasses BFV encryption, ZKP generation, and FHS signing, across four representative platforms: an Android Qualcomm Snapdragon 8 Gen 2 (aarch64) via PRoot Distro⁴ on Termux, an Apple M2 (macOS), a Dell Precision 3581 equipped with an Intel Core i7-1355U (Windows 11), the same Intel Core i7-1355U with different RAM running Ubuntu 22.04. As shown in Table 2, ballot creation time is primarily determined by the vote-encoding length $k = \lceil \log_2 n_{\text{canid}} \rceil$ augmented by the fixed $\kappa = 16$ credential bits, which shows that the per-ballot cost of Phoenix

⁴ <https://github.com/termux/proot-distro>

$\text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}, \text{Recover}}^{\text{du-mb-BPRIV}, \beta}(\lambda)$	$\mathcal{O}_{\text{voteR}}(id, v_0, v_1)$
1: $Checked, Happy \leftarrow \emptyset; V, PU, U, CU \leftarrow \text{empty}$ 2: $(pd, sd) \leftarrow \text{Setup}(1^\lambda)$ 3: for $id \in \mathcal{I}$ do 4: $(pc, sc) \leftarrow \text{Register}(id); U[id] \leftarrow sc; PU[id] \leftarrow pc$ 5: if $id \in \mathcal{D}$ then $CU[id] \leftarrow U[id]$ 6: $BB \leftarrow \mathcal{A}^{\mathcal{O}_{\text{voteR}}, \mathcal{O}_{\text{board}}}(pd, CU, PU, \mathcal{H}_{\text{check}})$ 7: if $\mathcal{H}_{\text{check}} \not\subseteq V \vee \text{ValidBoard}(BB, pd) = \perp$ then return $d \leftarrow \{0, 1\}$ 8: $d^* \leftarrow \mathcal{A}()$ 9: $(r^*, \Pi^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{tally}}}()$ 10: if $\text{VerifyTally}((pd, pbb, r^*), \Pi^*) = \perp$ then return $d \leftarrow \{0, 1\}$ 11: $d \leftarrow \mathcal{A}^{\mathcal{O}_{\text{verify}}}()$ 12: if $\mathcal{H}_{\text{check}} \not\subseteq Checked$ then return $d \leftarrow \{0, 1\}$ 13: if $\mathcal{H}_{\text{check}} \not\subseteq Happy$ then return d^* 14: return d	1: if $id \notin \mathcal{H}$ then return $\perp$ 2: for $i \in \{0, 1\}$ do $(pc, b_i, st_{\text{pre}}^i, st_{\text{post}}^i) \leftarrow \text{Vote}(pd, pc, v_i)$ 3: $V[id] \leftarrow V[id] \parallel (st_{\text{pre}}^0, st_{\text{post}}^0, v_0)$ 4: $BB_0 \leftarrow BB_0 \parallel (id, (pc, b_0)); BB_1 \leftarrow BB_1 \parallel (id, (pc, b_1))$ $\mathcal{O}_{\text{tally}}()$ 1: if $\beta = 0$ then $(r, \Pi) \leftarrow \text{Tally}(BB, pd, sd); \text{return } (r, \Pi)$ 2: $BB' \leftarrow \text{Recover}(BB, BB_0, BB_1)$ 3: $(r, \Pi) \leftarrow \text{Tally}(BB', pd, sd)$ 4: return $(r, \text{Sim}(pd, \text{Publish}(BB), r))$ $\mathcal{O}_{\text{verify}}(id)$ for $id \in \mathcal{H}_{\text{check}}$ 1: $Checked \leftarrow Checked \cup \{id\}$ 2: if $\text{VerifyVote}(id, st_{\text{pre}}, st_{\text{post}}, BB, pc, sc) = \top$ then 3: $Happy \leftarrow Happy \cup \{id\}$ 4: return $Happy$
$\mathcal{O}_{\text{board}}()$	
1: return $\text{Publish}(BB_\beta)$	

Fig. 2: The du-mb-BPRIV security experiment. $\mathcal{I} = \mathcal{H} \cup \mathcal{D}$ with $\mathcal{H}_{\text{check}} \subseteq \mathcal{H}$.

grows linearly in k . These sub-70 ms timings show Phoenix is practical even on mobile hardware for elections with up to 2^{25} candidates ($k=25$), with no inherent constraint on the number of voters as ballots are created independently at fixed per-voter cost.

	Dell Precision	M2	Core i7	Snapdragon
2^1 Candidates	2.719	2.368	2.631	2.894
2^5 Candidates	11.817	9.749	10.931	12.120
2^{10} Candidates	22.099	19.385	20.737	22.996
2^{25} Candidates	58.713	48.748	54.841	68.560

Table 2: Time in terms of milliseconds (averaged over three runs) to encrypt, commit, and sign a ballot with a k -bit vote encoding (2^k candidates) and $\kappa=16$ credential bits.

4 Security Analysis of Phoenix

Ballot Privacy. We consider the strong correctness, strong consistency, and ideal functionality from [9]. The most fundamental voting privacy property that has been well-studied is BPRIV [9] (no entity can learn the content of ballots), and an stronger notion is mb-BPRIV [92] (ballot privacy against an adversary who controls the ballot box). du-mb-BPRIV is an equivalent of mb-BPRIV, where verification is delayed until after tallying [28], which we focus on it here.

Definition 7 (du-mb-BPRIV [28]). Let $\mathcal{V}$ be a voting system, Recover a recovery algorithm, and $\mathcal{I} = \mathcal{H} \cup \mathcal{D}$ a partition into honest and dishonest voters with $\mathcal{H}_{\text{check}} \subseteq \mathcal{H}$ the verifying subset. A selection function for integers $m, n \geq 1$ is a mapping $\pi: [n] \rightarrow [m] \cup (\{0, 1\}^* \times \{0, 1\}^*)$, inducing a board transformation $\bar{\pi}$; the recovery algorithm $\text{Recover}(BB, BB_1)$ returns such a π capturing how $\mathcal{A}$ constructed BB from BB_1 . We say $\mathcal{V}$ satisfies du-mb-BPRIV w.r.t. Recover if there exists an efficient simulator Sim such that for every efficient adversary $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \mathcal{V}, \text{Sim}}^{\text{du-mb-bpriv}}(\lambda) = \left| \Pr[\text{Exp}^0 = 1] - \Pr[\text{Exp}^1 = 1] \right| = \text{negl}(\lambda)$, where $\text{Exp}^\beta := \text{Exp}_{\mathcal{A}, \mathcal{V}, \text{Sim}, \text{Recover}}^{\text{du-mb-BPRIV}, \beta}(\lambda)$ is defined in Figure 2.

In the experiment, $\mathcal{O}_{\text{voteLR}}$ computes ballots for both v_0, v_1 , storing them in BB_0, BB_1 ; the voter state records st_{pre}^β (dependent on the secret bit) and st_{post}^0 (always from $\beta=0$, since recovery ensures the tally reflects the left board). After $\mathcal{A}$ outputs board BB , it makes an initial guess d^* before seeing the tally. $\mathcal{O}_{\text{tally}}$ (single call): for $\beta=0$ tallies BB directly; for $\beta=1$ recovers $BB' \leftarrow \text{Recover}(BB, BB_0, BB_1)$, tallies BB' , and returns the result with a simulated proof. $\mathcal{O}_{\text{verify}}$ runs VerifyVote for voters in $\mathcal{H}_{\text{check}}$. If any expected verification is missing or fails, the experiment returns d^* (or a random bit), ensuring the adversary gains no advantage from tampering that causes detectable complaints.

Due to space limitations, for other properties and complete formal proofs of Theorems in this work refer to Appendix of full version [93].

Theorem 1. *Let BFV be IND-CPA secure under $\text{RLWE}_{n,q,\chi}$, ZKP be computationally zero-knowledge and knowledge-sound under MSIS and MLWE, and H be collision-resistant. Then for every PPT adversary $\mathcal{A}$ making at most q_v queries to $\mathcal{O}_{\text{voteLR}}$, there exist a simulator Sim , a recovery algorithm Recover , and PPT reductions $\mathcal{B}_{\text{cpa}}$, $\mathcal{B}_{\text{zk}}$, $\mathcal{B}_{\text{fhs}}$, $\mathcal{B}_{\text{cr}}$ such that*

$$\text{Adv}_{\mathcal{A}, \text{Phoenix}, \text{Sim}}^{\text{du-mb-bpriv}}(\lambda) \leq q_v \cdot \text{Adv}_{\mathcal{B}_{\text{cpa}}, \text{BFV}}^{\text{ind-cpa}}(\lambda) + q_{v+1} \cdot \text{Adv}_{\mathcal{B}_{\text{zk}}, \text{ZKP}}^{\text{zk}}(\lambda) + \text{Adv}_{\mathcal{B}_{\text{cr}}, H}^{\text{cr}}(\lambda) + \text{negl}(\lambda).$$

Intuitively, Phoenix achieves du-mb-BPRIV because IND-CPA security of BFV renders the ciphertexts on the adversary's view of the board independent of the real votes, computational zero-knowledge of ZKP extends this indistinguishability to the accompanying well-formedness proofs (and hence, by composition, to the FHS signatures computed over them), and collision resistance of H guarantees that the hash-based individual verification check cannot be exploited to distinguish the two worlds.

Strong Consistency. This property demands that for any (adversarially produced) bulletin board for which each individual ballot is valid, the result provided by the Tally algorithm is the correct one. The original definition associated voter identities with ballots; here, we use their public credentials instead.

Definition 8. *A voting scheme is strongly consistent if there exists:*

- An extraction algorithm $\text{Extract}(b, \text{sk})$ that provides the upk with the value computed by decrypting of the ciphertext contained in the ballot b with the secret sk ; and
- A ballot validation algorithm $\text{ValidInd}(b, \text{pk})$ returns true iff ballot b is “well-formed” with respect to some notion of well-formedness determined in advance by election.

These algorithms must satisfy the following conditions:

- For any (pk, sk) obtained from $\text{Setup}(\lambda)$, and any $(\text{id}, v, \text{usk})$ with upk trivially computed from usk , if $b \leftarrow \text{Vote}(\text{id}, v, \text{pk}, \text{usk})$ then $\text{Extract}(b, \text{sk})$ returns (upk, v) with overwhelming probability.
- For any adversarially produced (BB, b) , if $\text{Valid}(BB, b, \text{pk})$ returns true, then $\text{ValidInd}(b, \text{pk})$ returns true as well.
- For any adversary $\mathcal{B}$ that returns a ballot box with ballots that satisfy ValidInd , the experiment $\text{Exp}_{\mathcal{B}, v, I}^{\text{consis}}(\lambda)$ specified in Figure 3 returns true with a probability negligible in the security parameter.

$\text{Exp}_{\mathcal{B}, \mathcal{V}, I}^{\text{consis}}(\lambda)$	$\text{Exp}_{\mathcal{B}, \mathcal{V}, I}^{\text{corr}}(\lambda)$
1: $\text{uL} \leftarrow \text{empty}$	1: $\text{uL} \leftarrow \text{empty}$
2: $(\text{pk}, \text{sk}) \leftarrow \text{Setup}(1^\lambda)$	2: $(\text{pk}, \text{sk}) \leftarrow \text{Setup}(1^\lambda)$
3: $\forall id \in I \text{ do } \text{uL}[id] \leftarrow \text{Register}(id)$	3: $\forall id \in I \text{ do } \text{uL}[id] \leftarrow \text{Register}(id)$
4: $\text{BB} \leftarrow \mathcal{B}(\text{pk}, \text{uL})$	4: $(id, v, \text{BB}) \leftarrow \mathcal{B}(\text{pk}, \text{uL})$
5: $e \leftarrow \forall b \in \text{BB}. \text{ValidInd}(b, \text{pk})$	5: $e \leftarrow \text{false}$
6: $(r, \Pi) \leftarrow \text{Tally}(\text{BB}, \text{sk})$	6: if $(id \in I)$ then
7: for i in $1.. \text{BB} $ do	7: $(\text{usk}, \text{upk}) \leftarrow \text{uL}[id]$
8: $\text{dbb}[i] \leftarrow \text{Extract}(\text{BB}[i], \text{sk})$	8: $e_1 \leftarrow \forall x. \forall y. (x, y, \star, \star) \in \text{BB} \implies$
9: $r' \leftarrow \text{Count} \circ \text{Policy}(\text{dbb})$	9: $(x, y) = (id, \text{upk}) \vee (x \neq id \wedge y \neq \text{upk})$
10: return $(r \neq r' \wedge e)$	10: $b \leftarrow \text{Vote}(id, v, \text{pk}, \text{usk})$
	11: $e_2 \leftarrow \text{Valid}(\text{BB}, b, \text{pk})$
	12: $e \leftarrow e_1 \wedge \neg e_2$
	13: return e

Fig. 3: Security experiments for strong consistency (left, $\text{Exp}^{\text{consis}}$) and strong correctness (right, Exp^{corr}). The adversary $\mathcal{B}$ wins $\text{Exp}^{\text{consis}}$ if tallying produces a result inconsistent with the extracted ballots despite all ballots being individually valid. The adversary wins Exp^{corr} if it can submit a ballot bulletin board that renders an honest voter's fresh ballot invalid.

Theorem 2. *Let BFV be correct and IND-CPA-secure under $\text{RLWE}_{n,q,\chi}$, let ZKP be complete and knowledge-sound under MSIS, and let FHS be existentially unforgeable under $\text{SIS}_{n,m,q,\beta}$. Define $\text{Extract}(b, \text{sk})$ to parse $b = (id, \text{upk}, c, \sigma, s)$ and return $(\text{upk}, \text{BFV}.\text{Dec}(\text{sk}, c))$, and $\text{ValidInd}(b, \text{pk})$ to return 1 iff $\text{ZKP}.\text{Verify}(\text{pp}, (c, \text{upk}), \sigma) = 1$ and $\text{FHS}.\text{Verify}(\text{upk}, id, (c, \sigma), s) = 1$. Then for every PPT adversary $\mathcal{B}$, we have $\Pr[\text{Exp}_{\mathcal{B}, \text{Phoenix}, \mathcal{I}}^{\text{consis}}(\lambda) = 1] \leq \text{Adv}_{\text{ZKP}}^{\text{ks}}(\lambda) + \text{negl}(\lambda)$, where the negligible term accounts for the probability that BFV decryption fails on a correctly encrypted plaintext.*

In Phoenix, if every $b \in \text{BB}$ satisfies ValidInd , knowledge soundness of ZKP guarantees each c encrypts a valid $v \in \{0, 1\}$, and unforgeability of FHS binds each (c, σ) to its registered upk . Correctness of BFV then makes sure Extract recovers (upk, v) , and additive homomorphism yields $r = \sum v_i = \text{Count}(\text{dbb})$ with overwhelming probability.

Strong Correctness. This is the guarantee that an honestly created ballot will not be rejected by a ballot validation algorithm.

Definition 9 (Strong Correctness). *A voting scheme $\mathcal{V}$ is strongly correct if the advantage of any efficient adversary $\mathcal{B}$, defined by $\mathcal{A}_{\mathcal{B}, \mathcal{V}, I}^{\text{corr}}(\lambda) = \Pr[\text{Exp}_{\mathcal{B}, \mathcal{V}, I}^{\text{corr}}(\lambda) = 1]$ (where $\text{Exp}_{\mathcal{B}, \mathcal{V}, I}^{\text{corr}}(\lambda)$ is defined in Figure 3) is negligible as a function of λ .*

Theorem 3. *Let BFV be correct, let ZKP be complete, and let FHS be correct (i.e. $\text{FHS}.\text{Verify}$ accepts honestly generated signatures with overwhelming probability). Then for every PPT adversary $\mathcal{B}$, $\text{Adv}_{\mathcal{B}, \text{Phoenix}, \mathcal{I}}^{\text{corr}}(\lambda) = \Pr[\text{Exp}_{\mathcal{B}, \text{Phoenix}, \mathcal{I}}^{\text{corr}}(\lambda) = 1] \leq \text{negl}(\lambda)$.*

In Phoenix, an honest ballot runs $\text{BFV}.\text{Enc}$, $\text{ZKP}.\text{Prove}$, and $\text{FHS}.\text{Sign}$ with correctly generated keys, so completeness and correctness of the respective primitives ensure $e_2 = e_3 = 1$. The credential-consistency check e_1 holds because the experiment enforces no conflicting (id, upk) pair unless $\mathcal{B}$ itself placed it. Hence $\text{Valid}(\text{BB}, b, \text{pk}) = 1$ with overwhelming probability.

Ideal Functionality. This property states the voting protocol execution is indistinguishable from interaction with a trusted party.

Definition 10 (Ideal Voting Functionality). Let $\mathcal{I}$ be a set of voter identities, $\mathbb{V}$ a set of possible votes, and $\rho: (\mathcal{I} \times \mathbb{V})^* \rightarrow \mathcal{R}$ a result function mapping sequences of identity-vote pairs to a result space $\mathcal{R}$. The ideal functionality $\mathcal{F}_{\text{voting}}(\rho)$, interacting with an environment $\mathfrak{E}$ (controlling honest voters) and an ideal-world adversary (simulator) $\mathcal{S}$ (acting on behalf of corrupted voters), maintains an initially empty list L of identity-vote pairs and proceeds as follows:

1. **Vote:** On input $\text{vote}(id, v)$ from $\mathfrak{E}$ or $\mathcal{S}$, append (id, v) to L and send $\text{ack}(id)$ to $\mathcal{S}$.
2. **Tally:** On input tally from $\mathcal{S}$, compute $r \leftarrow \rho(L)$ and return r to both $\mathfrak{E}$ and $\mathcal{S}$, then halt.

The full ideal-world execution, in which $\mathfrak{E}$, $\mathcal{S}$, and $\mathcal{F}_{\text{voting}}(\rho)$ interact and $\mathfrak{E}$ outputs a single bit, is denoted $\beta \leftarrow \text{idealexec}(\mathfrak{E} \parallel \mathcal{S} \parallel \mathcal{F}_{\text{voting}}(\rho))$.

Theorem 4. If Phoenix is *du-mb-BPRIV* (Theorem 1), strongly consistent (Theorem 2), and strongly correct (Theorem 3), then for every PPT environment $\mathfrak{E}$ there exists a PPT simulator $\mathcal{S}$ such that

$$|\Pr[\text{realexec}(\mathfrak{E} \parallel \mathcal{A} \parallel \text{Phoenix}) = 1] - \Pr[\text{idealexec}(\mathfrak{E} \parallel \mathcal{S} \parallel \mathcal{F}_{\text{voting}}(\rho)) = 1]| \leq \text{negl}(\lambda).$$

By the composition theorem of Cortier et al. [9], any scheme satisfying *mb-BPRIV*, strong correctness, and strong consistency realises $\mathcal{F}_{\text{voting}}(\rho)$: strong correctness lets $\mathcal{S}$ simulate the honest voting interface; strong consistency ensures the real-world tally matches ρ on extracted votes; and *mb-BPRIV* lets $\mathcal{S}$ simulate the adversary's view from r alone. Since *du-mb-BPRIV* implies *mb-BPRIV* and Phoenix satisfies all three properties by Theorems 1–3, the composition applies directly.

Verifiability. Generally, it comes from the fact that voters can verify their ballots were recorded and anyone can verify correct tallying, but we consider the *Strong Verifiability* notion by [10] here, which covers broader spectrum.

Definition 11 (Strong Verifiability). Let $\mathcal{V} = (\text{Setup}, \text{Credential}, \text{Vote}, \text{Validate}, \text{Box}, \text{VerifyVote}, \text{Tally}, \text{Verify})$ be a voting protocol with admissible vote space $\mathbb{V}$, result space $\mathcal{R}$, and result function $\rho: \mathbb{V}^* \rightarrow \mathcal{R}$ admitting partial tallying via a commutative operation $\oplus_R$, i.e. $\rho(S_1 \cup S_2) = \rho(S_1) \oplus_R \rho(S_2)$ for disjoint vote lists $S_1, S_2 \subseteq \mathbb{V}^*$. Both experiments maintain: a public credential set L ; a table U of registered tuples $(id, \text{upk}, \text{usk})$; a set CU of corrupted voter pairs (id, upk) ; a list $HVote$ of triples (id, v, b) from honest voting queries (last ballot per id only); and $Checked \subseteq HVote$ where $CheckedVoter(id, v, b) = 1$. The oracles are: $\mathcal{O}_{\text{reg}}(id)$: honestly generates $(\text{upk}, \text{usk}) \leftarrow \text{Credential}(1^\lambda, id)$, returns upk , updates L, U . $\mathcal{O}_{\text{corrupt}}(id)$: reveals (upk, usk) for registered id , updates CU . $\mathcal{O}_{\text{vote}}(id, v)$: for honest, uncorrupted id with $v \in \mathbb{V}$, returns $b \leftarrow \text{Vote}(id, \text{upk}, \text{usk}, v)$, updates $HVote$. $\mathcal{O}_{\text{cast}}(id, b)$: runs $\text{Box}(BB, b)$ on an honest bulletin board.

The experiments are given in Figure 4. In both, the adversary wins if it produces a verified tally that cannot be expressed as $\rho(\{v_i^E\}) \oplus_R \rho(\{v_i^A\}) \oplus_R \rho(\{v_i^B\})$, where $\{v_i^E\}$ are votes from $Checked$, $\{v_i^A\}$ are from some subset of $HVote \setminus Checked$, and $\{v_i^B\} \subseteq \mathbb{V}$ with $|\{v_i^B\}| \leq |CU|$. We say $\mathcal{V}$ achieves strong verifiability if for all PPT $\mathcal{A}$, both $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{ver-b}}(\lambda) = 1]$ and $\Pr[\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{ver-r}}(\lambda) = 1]$ are negligible, i.e. under the assumption that the registration authority and bulletin board are not simultaneously dishonest.

$\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{ver-b}}(\lambda)$	$\text{Exp}_{\mathcal{A}, \mathcal{V}}^{\text{ver-r}}(\lambda)$
1: $L, U, CU, HVote \leftarrow \emptyset$	1: $L, U, CU, HVote \leftarrow \emptyset$
2: $(pk, sk) \leftarrow \text{Setup}(1^\lambda)$	2: $(pk, sk) \leftarrow \text{Setup}(1^\lambda)$
3: $(BB, \rho, \Pi) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{reg}}, \mathcal{O}_{\text{cast}}, \mathcal{O}_{\text{vote}}}(pk)$	3: $(\rho, \Pi) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{vote}}, \mathcal{O}_{\text{cast}}}(pk)$
4: if $\text{Verify}(BB, \rho, \Pi) = 0 \vee \rho = \perp$ then return 0	4: if $\text{Verify}(BB, \rho, \Pi) = 0 \vee \rho = \perp$ then return 0
5: if $\exists \{(id_i^A, v_i^A, \star)\} \subseteq HVote \setminus \text{Checked}, \exists \{v_i^B\} \subseteq \mathbb{V}, \{v_i^B\} \leq CU $:	5: if $\exists \{(id_i^A, v_i^A, \star)\} \subseteq HVote \setminus \text{Checked}, \exists \{v_i^B\} \subseteq \mathbb{V}, \{v_i^B\} \leq CU $:
6: $\rho = \rho(\{v_i^B\}) \oplus_{RP}(\{v_i^A\}) \oplus_{RP}(\{v_i^B\})$ then return 0	6: $\rho = \rho(\{v_i^B\}) \oplus_{RP}(\{v_i^A\}) \oplus_{RP}(\{v_i^B\})$ then return 0
7: return 1	7: return 1

Fig. 4: $\text{Exp}^{\text{ver-b}}$ (left): $\mathcal{A}$ controls trustee, voters, and bulletin board; registrar is honest. $\text{Exp}^{\text{ver-r}}$ (right): $\mathcal{A}$ controls trustee, voters, and registrar; bulletin board is honest ($\mathcal{A}$ adds ballots only via $\mathcal{O}_{\text{cast}}$, does not output BB).

Theorem 5. *Let H be collision-resistant, let ZKP be knowledge-sound under MSIS, let FHS be existentially unforgeable under $\text{SIS}_{n,m,q,\beta}$, and let BFV be correct. Then for every PPT adversary $\mathcal{A}$, $\Pr[\text{Exp}_{\mathcal{A}, \text{Phoenix}}^{\text{ver-b}}(\lambda) = 1] \leq \text{Adv}_{\text{FHS}}^{\text{euf}}(\lambda) + \text{Adv}_{\text{ZKP}}^{\text{ks}}(\lambda) + \text{Adv}_H^{\text{cr}}(\lambda) + \text{negl}(\lambda)$, and likewise $\Pr[\text{Exp}_{\mathcal{A}, \text{Phoenix}}^{\text{ver-r}}(\lambda) = 1] \leq \text{negl}(\lambda)$ under the same assumptions.*

Individual verifiability follows from collision resistance of H and unforgeability of FHS: substituting or omitting a ballot requires a hash collision or a signature forgery under the voter's upk . Universal verifiability follows from knowledge soundness of ZKP: an accepting tally proof guarantees r is the correct aggregate. Eligibility verifiability follows from unforgeability of FHS: every accepted ballot carries a valid signature under a registered upk , bounding adversarial contributions to $|CU|$.

5 Efficient Context-Hiding Leveled FHS

In Phoenix we use fully homomorphic signature but existing FHS constructions face a gap: Gorbunov et al. [26] sign each element independently via $\mathbf{U}_i \leftarrow \text{SamPre}(\mathbf{A}, \mathbf{V}_i - x_i \mathbf{G}, \text{td})$, requiring N costly trapdoor inversions, but achieve context-hiding by re-randomising the evaluated signature through SampleRight . Luo et al. [27] reduce this to just two SampleLeft calls by encoding both bit values via $\text{encode}(k) = [k\mathbf{I}_n \mid \dots \mid k\mathbf{I}_n]\mathbf{G}_n$ and deriving each σ_i deterministically through Unpack , but return the evaluated signature without re-randomisation, potentially leaking individual data beyond the computation output.

We resolve this by presenting the first FHS that achieves both properties simultaneously and instantiate the FHS in Phoenix with this variant. Since our scheme operates on multiple datasets distinguished by tags τ , we adopt the multi-data syntax: a signer can sign arbitrarily many datasets under different tags, each tag τ producing a verification key vk via KeyGen , and the verifier must know the tag τ of the dataset on which the computation was performed.

5.1 Our Fully Homomorphic Signature Scheme

Built upon [27], our scheme uses *lattice trapdoors* $\text{LT} = (\text{GenTrap}, \text{SampleLeft}, \text{SampleRight})$ [90, 91], where GenTrap outputs $(\mathbf{A}^*, \mathbf{T}_{\mathbf{A}^*})$, SampleLeft samples a short preimage under $[\mathbf{A}^* \mid \mathbf{C}]$, and SampleRight re-randomises a solution into a fresh statistically independent short preimage; *vector encoding* $\text{VE} = (\text{encode}, \text{Unpack})$, where $\text{encode}(k)$ maps $k \in \{0, 1\}$ to a structured offset matrix and $\text{Unpack}(\mathbf{R}, i)$ extracts the i -th block; and *tag evaluation*

$\text{TE} = (\text{PubEval}, \text{TrapEval})$, where $\text{PubEval}_{(\tau, \nu, t)}$ derives the per-tag matrix $\mathbf{B}_\tau \in \mathbb{Z}_q^{n \times m}$ from $(\mathbf{B}, \mathbf{B}')$. The gadget matrix $\mathbf{G}_n \in \mathbb{Z}_q^{n \times m}$ [91] admits binary decomposition $\mathbf{G}_n^{-1}(\cdot)$, $\mathbf{P}_{x_i}$ is the permutation selecting the i -th gadget block, and the verification bound B is derived from $s \geq \|\mathbf{T}_{\mathbf{A}^*}\| \cdot \omega(\sqrt{\log m})$.

Definition 12 (Efficient Context-Hiding Levelled Fully Homomorphic Signature). Let $\lambda, \ell, d_{\max} \in \mathbb{N}$ be the security parameter, maximum dataset size, and maximum circuit depth. Let n, m, q be lattice parameters with $m \geq 2n \lceil \log q \rceil$ determined by $(\lambda, d_{\max})$ under $\text{SIS}_{n, m, q, \beta}$, $|\tau| \geq \lambda$ the tag bit-length, and $\tilde{\beta}_{\max} = \beta_{\max} \cdot p_{\text{extra}}(n, m, \log q)$ the context-hiding norm bound. The fully homomorphic signature scheme $\text{FHS} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Eval}, \text{Verify}, \text{Hide}, \text{HVerify})$ is given in Figure 5.

FHS.Setup$(1^\lambda, 1^{ \tau }, 1^\ell, 1^{d_{\max}})$ 1: $(\mathbf{A}^*, \mathbf{T}_{\mathbf{A}^*}) \leftarrow \text{GenTrap}(1^n, 1^m, q)$ 2: $\mathbf{A}, \mathbf{D}_0, \mathbf{D}_1, \mathbf{B}, \mathbf{B}' \leftarrow \mathbb{Z}_q^{n \times m}$ 3: $\mathbf{r}_0 \leftarrow \mathbb{S}\{0, 1\}^m; \quad \mathbf{z} \leftarrow \mathbf{A}^* \cdot \mathbf{r}_0 \in \mathbb{Z}_q^n$ 4: $\nu \leftarrow \lceil \log \lceil \log \lceil \tau \rceil \rceil \rceil; \quad t \leftarrow \lceil 2 \log 2 \lceil \tau \rceil \rceil$ 5: $pk \leftarrow (\mathbf{A}, \mathbf{A}^*, \mathbf{D}_0, \mathbf{D}_1, \mathbf{B}, \mathbf{B}', \mathbf{z}, \nu, t)$ 6: $sk \leftarrow \mathbf{T}_{\mathbf{A}^*}$ return (pk, sk) FHS.KeyGen(pk, τ) 1: $\mathbf{B}_\tau \leftarrow \text{PubEval}_{(\tau, \nu, t)}(\mathbf{B}, \mathbf{B}', h_{\mathbf{A}, \alpha, \beta})$ 2: $\mathbf{A}_\tau \leftarrow [\mathbf{A}^* \mathbf{A} + \mathbf{B}_\tau] \in \mathbb{Z}_q^{n \times 2m}$ return $vk \leftarrow (\mathbf{A}_\tau, \mathbf{D}_0, \mathbf{D}_1, \mathbf{z})$ FHS.Hide(vk, y, σ_f) 1: $(\mathbf{R}_f, y, f) \leftarrow \sigma_f$ 2: Recompute $\mathbf{A}_\tau, \mathbf{D}_f$ from vk, f 3: $\mathbf{H}_f \leftarrow [\mathbf{A}_\tau \mathbf{D}_f + (y-1)\mathbf{G}_n] \in \mathbb{Z}_q^{n \times 3m}$ return $\tilde{\sigma}_f \leftarrow \text{SampleRight}(\mathbf{H}_f, \mathbf{R}_f, \mathbf{z})$	FHS.Sign(sk, vk, τ, μ, i) 1: for $k \in \{0, 1\}$ do $\begin{bmatrix} \mathbf{R}_{k,1} \\ \mathbf{R}_{k,2} \end{bmatrix} \leftarrow \text{SampleLeft}(\mathbf{A}^*, \mathbf{A} + \mathbf{B}_\tau, \mathbf{T}_{\mathbf{A}^*},$ $\mathbf{D}_k + \text{encode}(k), s)$ 2: $\begin{bmatrix} \mathbf{R}_{\mu_i, 1, i} \\ \mathbf{R}_{\mu_i, 2, i} \end{bmatrix} \leftarrow \text{Unpack}\left(\begin{bmatrix} \mathbf{R}_{\mu_i, 1} \\ \mathbf{R}_{\mu_i, 2} \end{bmatrix}, i\right)$ return $\sigma_i \leftarrow \left(\begin{bmatrix} \mathbf{R}_{\mu_i, 1, i} \\ \mathbf{R}_{\mu_i, 2, i} \end{bmatrix}, \mu_i, i\right)$ FHS.Eval$(vk, \tau, \mu, \sigma, f)$ 1: for each NAND-gate $g = (u, v, w)$ in f do $\hat{\mathbf{D}}_u \leftarrow \mathbf{G}_n^{-1}(\mathbf{D}_u \mathbf{G}_n^{-1}(\mathbf{P}_{x_i} \mathbf{G}_n))$ $\mathbf{R}_w \leftarrow \mathbf{R}_u \hat{\mathbf{D}}_u - y \mathbf{R}_v$ $\mathbf{D}_w \leftarrow \mathbf{D}_v \mathbf{G}_n^{-1}(\mathbf{P}_{y_f} \mathbf{G}_n) \hat{\mathbf{D}}_u - \mathbf{G}_n$ return $\sigma_f \leftarrow (\mathbf{R}_f, f(\mu), f)$	FHS.Verify(vk, τ, σ_f, f) 1: $(\mathbf{R}_f, y, f) \leftarrow \sigma_f$ 2: Recompute $\mathbf{A}_\tau, \mathbf{D}_f$ from vk, τ, f 3: $e_1 \leftarrow \ \mathbf{R}_f\ _\infty \leq B$ 4: $e_2 \leftarrow (\mathbf{A}_\tau \mathbf{R}_f = \mathbf{D}_f + y \mathbf{G}_n)$ return $(e_1 \wedge e_2)$ FHS.HVerify$(vk, \tau, \tilde{\sigma}_f, y, f)$ 1: Recompute $\mathbf{A}_\tau, \mathbf{D}_f$ from vk, τ, f 2: $\mathbf{H}_f \leftarrow [\mathbf{A}_\tau \mathbf{D}_f + (y-1)\mathbf{G}_n]$ 3: $e_1 \leftarrow (\mathbf{H}_f \cdot \tilde{\sigma}_f = \mathbf{z})$ 4: $e_2 \leftarrow (\ \tilde{\sigma}_f\ _\infty \leq \tilde{\beta}_{\max})$ return $(e_1 \wedge e_2)$
---	--	--

Fig. 5: Our context-hiding leveled FHS, where a simulator $\tilde{\sigma}_f \leftarrow \text{Sim}(sk, vk, \tau, y, f)$ produces output indistinguishable from Hide by the SampleLeft - SampleRight statistical closeness (see Lemma 6.4 of [26]). The matrix $\mathbf{G}_n$ denotes the gadget matrix and $B, \tilde{\beta}_{\max}$ the verification and context-hiding norm bounds, respectively. $\mathbf{R}_{k,1}$ and $\mathbf{R}_{k,2}$ are left and right preimage matrix blocks.

Theorem 6. Under $\text{SIS}(n, m, q, \beta)$, for every PPT adversary $\mathcal{A}$, $\Pr[\text{Expt}_{\mathcal{A}, \text{FHS}}^{\text{EU-F-ACMA}}(1^\lambda) = 1] \leq \text{Adv}_{n, m, q, \beta}^{\text{sis}}(\lambda) + \text{negl}(\lambda)$, where FHS is the scheme of Definition 12 and $\text{Expt}_{\mathcal{A}, \text{FHS}}^{\text{EU-F-ACMA}}$ is the experiment of Definition 5.

Any winning adversary must produce $(\tau^*, \sigma_{f^*}^*, \mu^*, f^*)$ with $\text{Verify}(vk^*, \tau^*, \sigma_{f^*}^*, f^*) = 1$. In a Type I forgery, the extracted $\mathbf{R}_f^*$ satisfies $\mathbf{A}_{\tau^*} \mathbf{R}_f^* = \mathbf{D}_{f^*} + \mu^* \mathbf{G}_n$ with $\|\mathbf{R}_f^*\|_\infty \leq B$ on a never-queried tag, yielding a short SIS solution directly. In a Type II forgery, subtracting the verification equations for two valid signatures on the same tag τ_j but distinct outputs produces a non-trivial short kernel vector, again breaking $\text{SIS}_{n, m, q, \beta}$.

Theorem 7. The FHS satisfies Definition 6: for every (unbounded) adversary $\mathcal{A}$, every $(pk, sk) \leftarrow \text{Setup}(1^\lambda)$, $vk \leftarrow \text{KeyGen}(pk, \tau)$, and every (σ_f, y, f) with $\text{Verify}(vk, \tau, \sigma_f, f) = 1$, $\Delta(\text{Hide}(vk, y, \sigma_f), \text{Sim}(sk, vk, \tau, y, f)) \leq \text{negl}(\lambda)$, where $\Delta(\cdot, \cdot)$ denotes statistical distance, following from the SampleLeft - SampleRight statistical closeness of [26].

For any valid $\sigma_f = (\mathbf{R}_f, y, f)$, Hide calls $\text{SampleRight}(\mathbf{H}_f, \mathbf{R}_f, \mathbf{z})$ with $\mathbf{H}_f = [\mathbf{A}_\tau \mid \mathbf{D}_f + (y-1)\mathbf{G}_n]$, while $\text{Sim}(sk, vk, \tau, y, f)$ recomputes the same $\mathbf{H}_f$ and calls SampleLeft from the trapdoor without accessing $\mathbf{R}_f$. The SampleLeft – SampleRight statistical closeness of [26] implies $\Delta(\text{Hide}(vk, y, \sigma_f), \text{Sim}(sk, vk, \tau, y, f)) \leq \text{negl}(\lambda)$, so $\tilde{\sigma}_f$ reveals nothing beyond y .

5.2 Efficiency Comparison with Gurbanov et al. [26]

Here, we discuss how our scheme’s time and space cost is more efficient than Gurbanov et al. [26], even by adding context-hiding property.

Signing complexity. In [26], Sign invokes SamPre once per data element $i \in [N]$, costing $O(N \cdot m^2 n)$ time and $O(N \cdot m^2 \log q)$ bits of storage. Our FHS calls SampleLeft exactly twice (for $k \in \{0, 1\}$), regardless of ℓ ; each per-message signature is then derived via the deterministic multiplication $[\mathbf{R}_{\mu_i, 1, i}^\top \mid \mathbf{R}_{\mu_i, 2, i}^\top]^\top = [\mathbf{R}_{k, 1}^\top \mid \mathbf{R}_{k, 2}^\top]^\top \cdot \mathbf{G}_n^{-1}(\mathbf{P}_i \mathbf{G}_n)$ at $O(m^2)$ per message (since $\mathbf{G}_n^{-1}(\mathbf{P}_i \mathbf{G}_n) \in \{0, 1\}^{m \times m}$), giving total cost $O((\ell + n) \cdot m^2)$ —a factor- $\Theta(n)$ improvement for $\ell \gg 1$.

Context-hiding overhead and space cost. Both schemes achieve statistical context hiding via a single SampleRight/SampleLeft call, adding $\tilde{O}(m^2 n)$ overhead. Our hiding matrix $\mathbf{H}_f = [\mathbf{A}_\tau \mid \mathbf{D}_f + (y-1)\mathbf{G}_n] \in \mathbb{Z}_q^{n \times 3m}$ yields $\tilde{\sigma}_f \in \mathbb{Z}^{3m}$ versus $\tilde{\sigma} \in \mathbb{Z}^{2m}$ in [26]—a modest constant-factor increase. For public-key size, [26] requires $O(N \cdot nm \log q)$ bits for $(\mathbf{V}_1, \dots, \mathbf{V}_N)$, while our $pk = (\mathbf{A}, \mathbf{A}^*, \mathbf{D}_0, \mathbf{D}_1, \mathbf{B}, \mathbf{B}', \mathbf{z})$ occupies only $O(nm \log q)$ bits independent of ℓ , with $\mathbf{z} \in \mathbb{Z}_q^n$ adding a negligible $O(n \log q)$ bits. Secret keys are identical in both ($O(m^2 \log q)$). Overall, our FHS achieves two pre-image samples instead of ℓ , dataset-independent public keys, and context hiding—at the cost of only one extra element $\mathbf{z}$ and a hidden signature of size $3m$ versus $2m$.

Future Directions. An FHS is used in Phoenix instead of a classical signature scheme because unlike other post-quantum schemes [4, 14–23], we plan to deploy an actual production-ready implementation of Phoenix as an online web-application voting platform, which in next update, the context-hiding property of FHS [94], along with using fake credentials [95], will get used to achieve stronger privacy guarantees, such as receipt-freeness [96] (voters cannot construct proofs of their choices to third parties) and coercion-resistance [97] (the coercer cannot know whether voter followed his instruction or not).

We aim to eliminate the distributed key generation phase on real-world deployment of Phoenix via silent-setup threshold cryptosystems [98]; however, current constructions rely on witness encryption [99], which remains inefficient and quantum-vulnerable [100, 101]. Designing a post-quantum *silent-setup* threshold public key scheme that can support homomorphic operations remains an open problem. In addition, except using signatures on randomizable ciphertexts [102], no solution has been proposed to achieve strong receipt-freeness [96], so using the same technique on PQ voting can be progressive, but designing post-quantum signatures on randomizable ciphertexts remains as an open problem.

References

1. M. Bernhard et al. Public evidence from secret ballots. In *Electronic Voting*, LNCS, pages 84–109. Springer, 2017.
2. J. Budurushi et al. An investigation into the usability of electronic voting systems for complex elections. *Ann. Telecommun.*, 71(7–8):309–322, 2016.
3. D. Rausch, N. Huber, and R. Küsters. Verifiable e-voting with a trustless bulletin board. In *IEEE CSF*, pages 457–472, 2025.
4. G. Kaim et al. Post-quantum online voting scheme. In *FC Workshops*, pages 290–305. Springer, 2021.
5. J. Mascelli and M. Rodden. “harvest now decrypt later”: Examining post-quantum cryptography and the data privacy risks for distributed ledger networks. Technical Report FEDS 2025-093, Board of Governors of the Federal Reserve System, 2025.
6. D.J. Bernstein and T. Lange. Post-quantum cryptography. *Nature*, 549(7671):188–194, 2017.
7. D. Sikeridis, P. Kampanakis, and M. Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. In *NDSS*, 2020.
8. A. Aydeger et al. Post-quantum public key infrastructures: Hybrid certificates, cryptographic combiners, and migration strategies. In *IEEE CCNC*, pages 1–4, 2026.
9. D. Bernhard et al. Sok: A comprehensive analysis of game-based ballot privacy definitions. In *IEEE S&P*, pages 499–516, 2015.
10. V. Cortier et al. Election verifiability for Helios under weaker trust assumptions. In *ESORICS*, pages 327–344. Springer, 2014.
11. V. Cortier et al. Election verifiability for Helios under weaker trust assumptions. In *ESORICS*, volume 8713 of *LNCS*, pages 327–344. Springer, 2014.
12. V. Cortier et al. Sok: Verifiability notions for e-voting protocols. In *IEEE S&P*, pages 779–798, 2016.
13. R. Küsters, T. Truderung, and A. Vogt. Accountability: Definition and relationship to verifiability. In *ACM CCS*, pages 526–535, 2010.
14. X. Boyen, T. Haines, and J. Müller. Epoque: Practical end-to-end verifiable post-quantum-secure e-voting. In *IEEE EuroS&P*, pages 272–291, 2021.
15. P. Hough, C. Sandsbråten, and T. Silde. More efficient lattice-based electronic voting from NTRU. *IACR Commun. Cryptol.*, 1(4), 2025.
16. R. Mosaheb. *Quantum-Safe Electronic Voting Schemes*. PhD thesis, Université du Luxembourg, 2025.
17. R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
18. I. Black et al. Practical quantum-safe voting from lattices, extended. Cryptology ePrint Archive, Paper 2022/1686, 2022.
19. I. Chillotti et al. A homomorphic LWE based e-voting scheme. In *PQCrypto*, pages 245–265. Springer, 2016.
20. N. Abapour et al. Blockchain-based economic voting with posterior security from lattices. In *IWSEC*, pages 247–266. Springer, 2025.
21. P. Sekar et al. Quantum-resistant electronic voting system using lattice cryptography techniques. In *Internet and Modern Society*, pages 257–270. Springer, 2025.
22. S.P. Mahanayak, B. Nikhita, and S. Bilgaiyan. Enhancing e-voting security with quantum-resistant encryption. *SN Comput. Sci.*, 4(5), 2023.
23. A. Junior Gabriel et al. Post-quantum cryptography system for secure electronic voting. *Open Comput. Sci.*, 9(1):292–298, 2019.
24. D.F. Aranha et al. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. In *ACM CCS*, pages 1467–1481, 2023.
25. F. Hao et al. Experience from UNITA elections: Reconciling revote, E2E verifiability and low coercion. Cryptology ePrint Archive, Paper 2025/1669, 2025.

26. S. Gorbunov, V. Vaikuntanathan, and D. Wichs. Leveled fully homomorphic signatures from standard lattices. In *ACM STOC*, pages 469–477, 2015.
27. F. Luo et al. A more efficient leveled strongly-unforgeable fully homomorphic signature scheme. *Inf. Sci.*, 480:70–89, 2019.
28. C.C. Drăgan et al. Machine-checked proofs of privacy against malicious boards for Selene & co. In *IEEE CSF*, pages 335–347, 2022.
29. Anonymous github. <https://anonymous.4open.science/r/phoenix1>. Accessed: 2026-03-25.
30. D.F. Aranha et al. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. Cryptology ePrint Archive, Paper 2022/422, 2022.
31. X. Boyen, T. Haines, and J. Mueller. Epoque: Practical end-to-end verifiable post-quantum-secure e-voting. Cryptology ePrint Archive, Paper 2021/304, 2021.
32. R. Küsters, T. Truderung, and A. Vogt. Verifiability, privacy, and coercion-resistance: New insights from a case study. In *IEEE S&P*, pages 538–553, 2011.
33. J. Benaloh. Verifiable secret-ballot elections. In *Ph.D. Thesis, Yale University*, 1987.
34. J. Benaloh et al. ElectionGuard: A cryptographic toolkit to enable verifiable elections. In *USENIX Security*, pages 5485–5502, 2024.
35. S. Bouaziz-Ermann et al. Lattice-based (partially) blind signature without restart. Cryptology ePrint Archive, Report 2020/260, 2020.
36. D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT*, volume 7237 of *LNCS*, pages 700–718. Springer, 2012.
37. R. Bendlin, S. Krehbiel, and C. Peikert. How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE. In *ACNS*, volume 7954 of *LNCS*, pages 218–236. Springer, 2013.
38. C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *ACM STOC*, pages 197–206, 2008.
39. V. Lyubashevsky, C. Peikert, and O. Regev. A toolkit for ring-LWE cryptography. In *EUROCRYPT*, volume 7881 of *LNCS*, pages 35–54. Springer, 2013.
40. E. Hauck et al. Lattice-based blind signatures, revisited. In *CRYPTO*, volume 12171 of *LNCS*. Springer, 2020.
41. A. Esser, A. May, and F. Zwegdinger. McEliece needs a break – solving McEliece-1284 and quasi-cyclic-2918 with modern ISD. In *EUROCRYPT*, volume 13277 of *LNCS*, pages 433–457. Springer, 2022.
42. S. Park et al. Going from bad to worse: From internet voting to blockchain voting. *J. Cybersecur.*, 7(1), 2021.
43. S. Agrawal, D. Boneh, and X. Boyen. Efficient lattice (H)IBE in the standard model. In *EUROCRYPT*, pages 553–572, 2010.
44. R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
45. R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
46. D. Stehlé and R. Steinfeld. Making NTRU as secure as worst-case problems over ideal lattices. In *EUROCRYPT*, volume 6632 of *LNCS*, pages 27–47. Springer, 2011.
47. C. Baum et al. More efficient commitments from structured lattice assumptions. In *SCN*, volume 11035 of *LNCS*, pages 368–385. Springer, 2018.
48. D.F. Aranha et al. Lattice-based proof of shuffle and applications to electronic voting. In *CT-RSA*, volume 12704 of *LNCS*, pages 227–251. Springer, 2021.
49. D.F. Aranha et al. Verifiable mix-nets and distributed decryption for voting from lattice-based assumptions. In *ACM CCS*, pages 1467–1481, 2023.
50. J. Bootle et al. More efficient amortization of exact zero-knowledge proofs for LWE. In *ESORICS*, volume 12973 of *LNCS*, pages 608–627. Springer, 2021.
51. N. Aragon et al. BIKE: Bit-flipping key encapsulation. https://bikesuite.org/files/v5.0/BIKE_Spec.2022.10.10.1.pdf, 2022.

52. C. Aguilar-Melchor et al. Efficient encryption from random quasi-cyclic codes. *IEEE Trans. Inf. Theory*, 64(5):3927–3943, 2018.
53. E. Bellini et al. Enhancing code based zero-knowledge proofs using rank metric. In *CANS*, volume 12579, pages 570–592, 2020.
54. F. Benhamouda et al. Better zero-knowledge proofs for lattice encryption and their application to group signatures. In *ASIACRYPT*, pages 551–572, 2014.
55. V. Lyubashevsky. Lattice signatures without trapdoors. In *EUROCRYPT*, pages 738–755, 2012.
56. C. Baum et al. Efficient commitments and zero-knowledge protocols from ring-SIS with applications to lattice-based threshold cryptosystems. *Cryptology ePrint Archive* 2016/997, 2016.
57. R. del Pino et al. Practical quantum-safe voting from lattices. In *ACM CCS*, pages 1565–1581, 2017.
58. L. Ducas and D. Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In *EUROCRYPT*, volume 9056 of *LNCS*, pages 617–640. Springer, 2015.
59. L. Ducas et al. Lattice signatures and bimodal gaussians. In *CRYPTO*, volume 8042 of *LNCS*, pages 40–56. Springer, 2013.
60. F. Laguillaumie et al. Lattice-based group signatures with logarithmic signature size. In *ASIACRYPT*, volume 8270 of *LNCS*, pages 41–61. Springer, 2013.
61. I. Hwang, J. Seo, and Y. Song. Concretely efficient lattice-based polynomial commitment from standard assumptions. In *CRYPTO*, pages 414–448. Springer, 2024.
62. T.H. Yuen et al. Posterior security: Anonymity and message hiding of standard signatures. In *ACM CCS*, pages 528–542, 2025.
63. F. Baldimtsi et al. Non-interactive blind signatures: Post-quantum and stronger security. In *ASIACRYPT*, volume 15485 of *LNCS*. Springer, 2025.
64. J. Bos et al. CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM. *Cryptology ePrint Archive*, Report 2017/634, 2017.
65. J. Huang et al. The application of the blockchain technology in voting systems. *ACM Comput. Surv.*, 54:1–28, 2021.
66. S. Vivek et al. E-voting systems using blockchain: An exploratory literature survey. In *IEEE ICIRCA*, 2020.
67. National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, NIST, 2015.
68. W. Diffie and M.E. Hellman. New directions in cryptography. *IEEE Trans. Inf. Theory*, 22(6):644–654, 1976.
69. B. Guidi, A. Michienzi, and L. Ricci. Data persistence in decentralized social applications: the IPFS approach. In *IEEE CCNC*, 2021.
70. Y. Lin and C. Zhang. A method for protecting private data in IPFS. In *IEEE CSCWD*, 2021.
71. A.J. Gabriel et al. Post-quantum cryptography: A combination of post-quantum cryptography and steganography. In *ICITST*, pages 454–457, 2013.
72. A. Petzoldt. *Selecting and Reducing Key Sizes for Multivariate Cryptography*. PhD thesis, Technical University Darmstadt, 2013.
73. Z. Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In *CRYPTO*, pages 868–886. Springer, 2012.
74. V. Lyubashevsky, N.K. Nguyen, and M. Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In *CRYPTO*, pages 71–101. Springer, 2022.
75. M. Ajtai. Generating hard instances of lattice problems (extended abstract). In *ACM STOC*, pages 99–108, 1996.
76. A. Langlois and D. Stehle. Worst-case to average-case reductions for module lattices. *Des. Codes Cryptogr.*, 75(3):565–599, 2015.

77. O. Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), 2009.
78. Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Trans. Comput. Theory*, 6(3), 2014.
79. National Institute of Standards and Technology. Secure hash standard (SHS). Technical Report FIPS PUB 180-4, NIST, 2015.
80. J. Fan and F. Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.
81. V. Lyubashevsky, C. Peikert, and O. Regev. On ideal lattices and learning with errors over rings. In *EUROCRYPT*, volume 6110 of *LNCS*, pages 1–23. Springer, 2010.
82. C. Peikert and S. Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In *CRYPTO*, pages 89–114. Springer, 2019.
83. A. Chailloux et al. Interactive and noninteractive zero knowledge are equivalent in the help model. In *TCC*, volume 4948 of *LNCS*. Springer, 2008.
84. J. Groth, R. Ostrovsky, and A. Sahai. Perfect non-interactive zero knowledge for NP. In *EUROCRYPT*, volume 4004 of *LNCS*. Springer, 2006.
85. R. Gay and B. Ursu. On instantiating unleveled fully-homomorphic signatures from falsifiable assumptions. In *PKC*, volume 14601 of *LNCS*. Springer, 2024.
86. D. Boneh et al. Threshold cryptosystems from threshold fully homomorphic encryption. In *CRYPTO*, pages 565–596. Springer, 2018.
87. D.J. Bernstein. Cost analysis of hash collisions: Will quantum computers make SHARCS obsolete? Workshop Record of SHARCS’09, 2009.
88. M. Albrecht et al. Homomorphic encryption security standard. HomomorphicEncryption.org, 2018.
89. M.R. Albrecht, R. Player, and S. Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 9(3):169–203, 2015.
90. C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *ACM STOC*, pages 197–206, 2008.
91. D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT*, volume 7237 of *LNCS*, pages 700–718. Springer, 2012.
92. V. Cortier, J. Lallemand, and B. Warinschi. Fifty shades of ballot privacy: Privacy against a malicious board. In *IEEE CSF*, pages 17–32, 2020.
93. Anonymous Github — anonymous.4open.science. <https://anonymous.4open.science/r/phoenix2/>. [Accessed 03-04-2026].
94. N. Attrapadung, B. Libert, and T. Peters. Efficient completely context-hiding quotable and linearly homomorphic signatures. In *PKC*, volume 7778 of *LNCS*. Springer, 2013.
95. Véronique Cortier, Pierrick Gaudry, and Quentin Yang. Is the jcy voting system really coercion-resistant? In *2024 IEEE 37th Computer Security Foundations Symposium (CSF)*, pages 186–200, 2024.
96. S. Delaune, S. Kremer, and M. Ryan. Coercion-resistance and receipt-freeness in electronic voting. In *IEEE CSFW*, pages 28–42, 2006.
97. T. Haines and B. Smyth. Surveying definitions of coercion resistance. Cryptology ePrint Archive, Paper 2019/822, 2019.
98. S. Garg et al. Threshold encryption with silent setup. In *CRYPTO*, pages 352–386. Springer, 2024.
99. B. Waters and D.J. Wu. Silent threshold cryptography from pairings: Expressive policies in the plain model. Cryptology ePrint Archive, Paper 2025/1547, 2025.
100. M. Hall-Andersen, M. Simkin, and B. Wagner. Silent threshold encryption with one-shot adaptive security. Cryptology ePrint Archive, Paper 2025/1384, 2025.
101. J. Bormet et al. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Paper 2025/1419, 2025.
102. P. Chaidos et al. BeleniosRF: A non-interactive receipt-free electronic voting scheme. In *ACM CCS*, pages 1614–1625, 2016.